

IP Model Automation — Pipeline Overview

This document explains what this project does and how the flow works, end to end: from a hardware IP design document (DLD) to a validated SimPy performance model with unit tests.

1. What we are doing

Hardware teams give us DLDs (Design-Level Documents): human-written markdown describing an IP block — its interfaces, internal state machines (FSMs), timing, and behavior. We need a SimPy transaction-level delay model and unit tests for each IP. Two problems make this hard:

- DLDs are inconsistent — every author formats things differently.
- DLDs omit details — command encodings, test scenarios, and exact invariants are often missing.

So we do not generate models directly from the DLD. Instead we introduce a template as a normalized, machine-checkable golden reference in the middle:

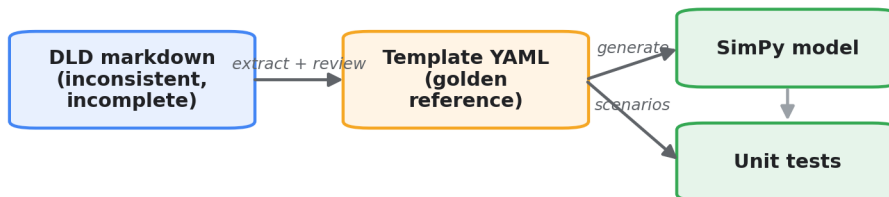


Figure 1. High-level flow: DLD to template to model and tests.

The key rule: two sources of truth, one at a time

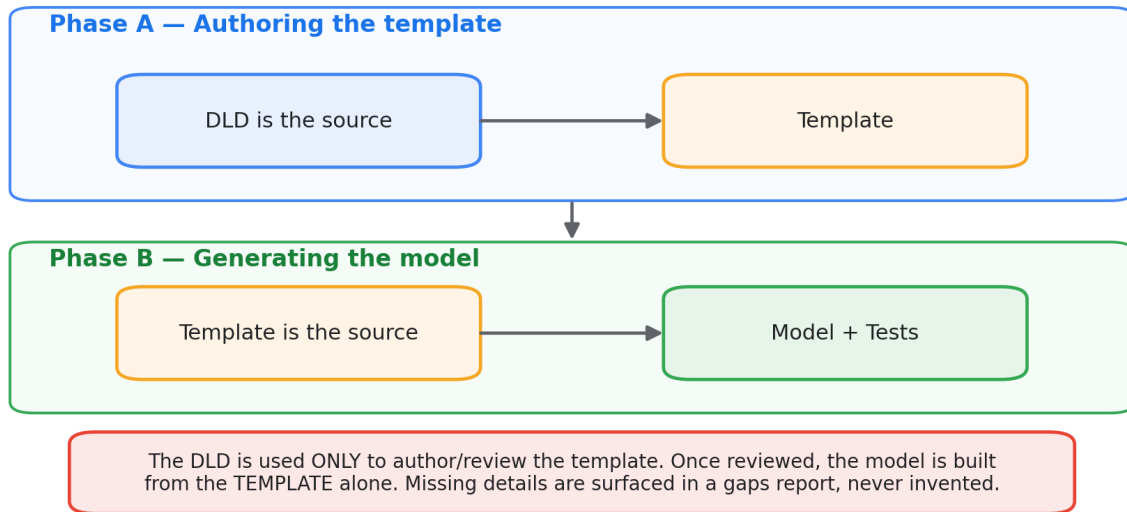


Figure 2. The DLD authors the template; the template alone generates the model.

This separation keeps generated models trustworthy: a reviewer signs off the template once, and everything downstream is a faithful function of it.

2. The end-to-end flow

Every stage is guarded by an automated gate. Nothing advances until its gate is green.

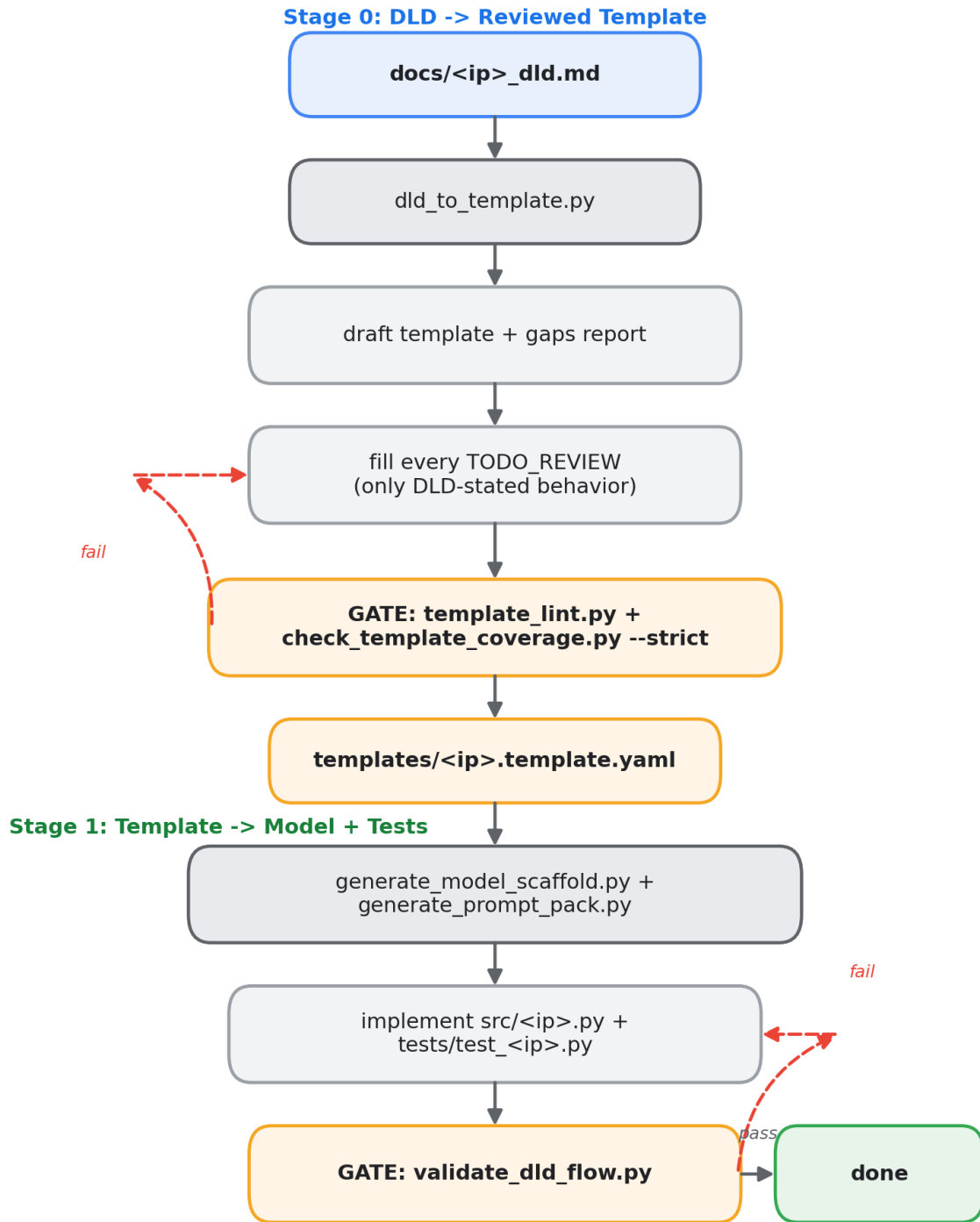


Figure 3. The full pipeline, from DLD to validated model, with gates and feedback loops.

3. Stage 0 in detail — turning a messy DLD into a clean template

tools/dld_to_template.py is a best-effort extractor. It parses the DLD's markdown conventions and fills what it can; anything it cannot derive is marked TODO_REVIEW and listed in a gaps report.

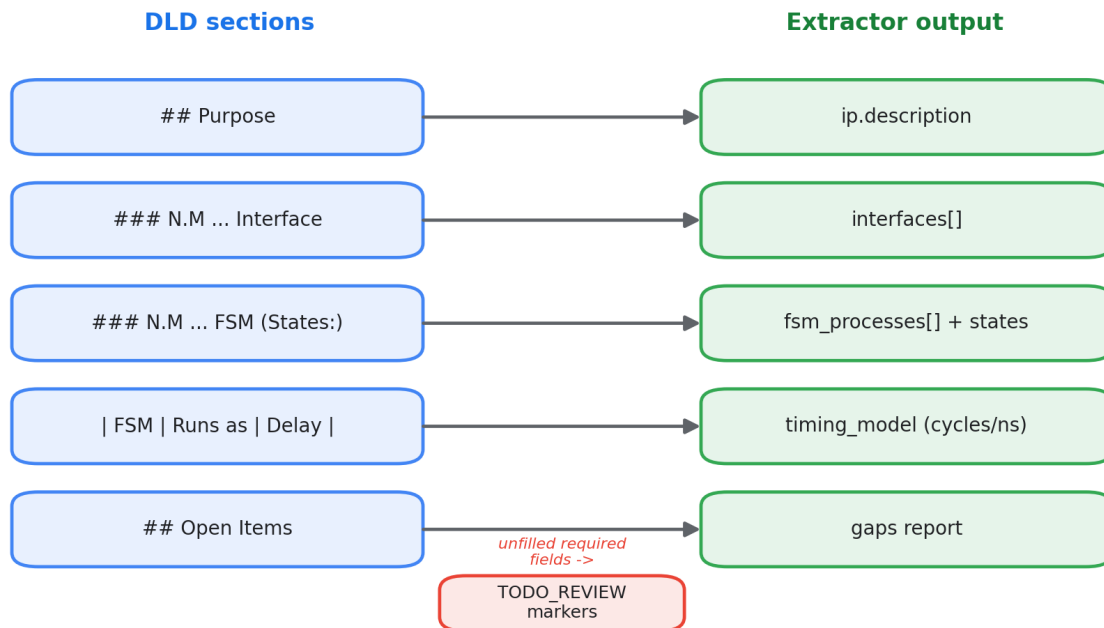


Figure 4. How DLD sections map to template fields, with unfilled fields flagged.

- Reliably extracted: FSM names + states, interface sections, per-FSM timing operations, clock rate, and FSM count.
- Left for review: commands, test scenarios, invariants, transition detail, and any IP-specific contract fields.
- The gaps report makes missing information visible and tracked, rather than silently filled with a plausible-but-wrong assumption.

4. The gates (what "validated" means)

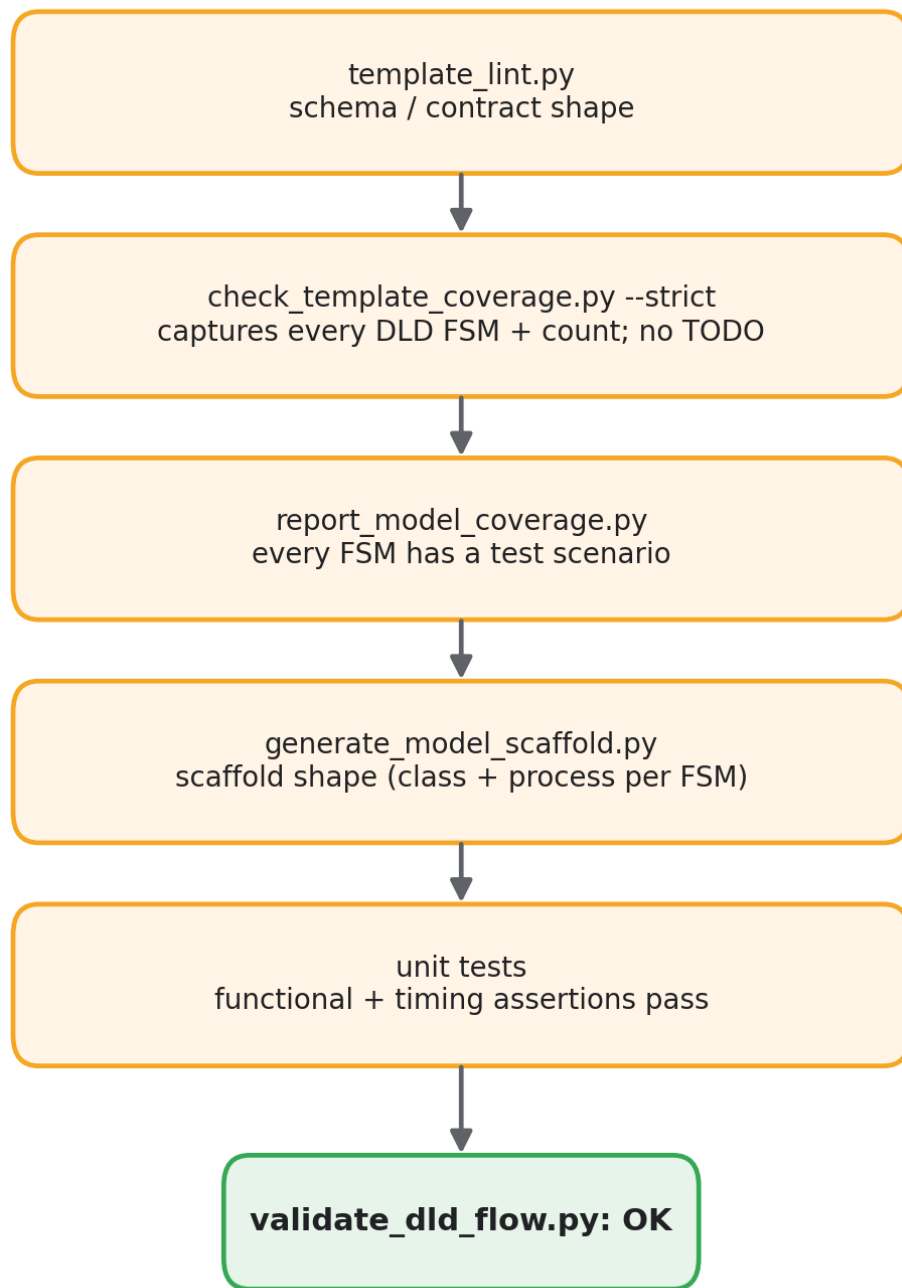


Figure 5. The gate chain. `validate_dld_flow.py` runs all of them across every IP.

Calibration — how we trust the extractor

The extractor was tuned against the existing golden IPs: re-parsing each DLD reproduces its template's FSM name set exactly, and the DLD's stated "Total FSM/processes: N" matches the template's `fsm_count`. That equality is the regression gate — if a future change breaks extraction,

calibration fails loudly. Interface sections are report-only, because templates legitimately consolidate them; FSM coverage is the hard gate.

5. Repository components

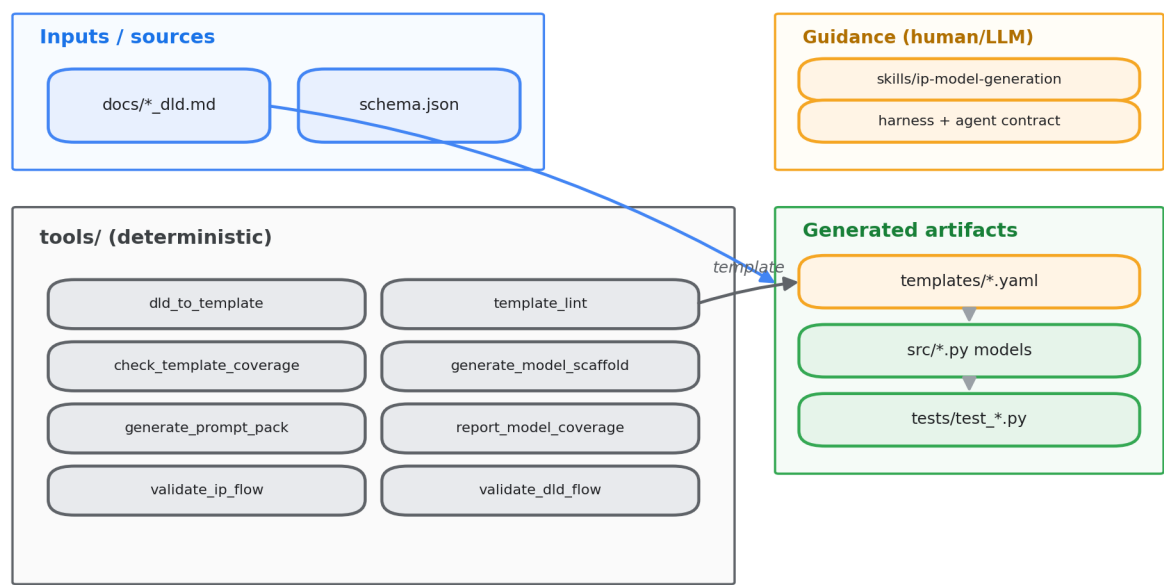


Figure 6. Repository layout: inputs, deterministic tools, guidance, and generated artifacts.

Area	Role
<code>docs/*_dld.md</code>	Human design intent (authoring source only).
<code>templates/*.template.yaml</code>	Golden reference — source of truth for generation.
<code>schemas/...schema.json</code>	Machine-checkable template contract.
<code>tools/*.py</code>	Deterministic extraction, linting, coverage, generation, validation.
<code>skills / harness / agents</code>	Instructions and contract for the human or LLM that fills templates, models, and tests.
<code>src/ip_model_automation/* .py</code>	Flat, one-file-per-IP SimPy delay models.
<code>tests/test_*.py</code>	Per-IP unit tests derived from <code>test_scenarios</code> .

6. What a generated model looks like

Each template FSM becomes one SimPy process; queues/stores connect them. Example — the Mailbox IP, one of two IPs added through this pipeline:

Mailbox IP — five parallel FSM processes + queues

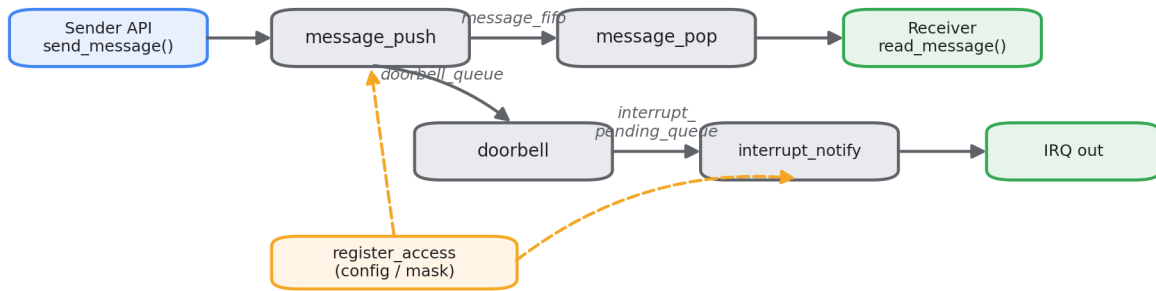


Figure 7. Mailbox IP internal structure: five parallel FSM processes and their queues.

- Concurrency: all five FSMs run as parallel `env.process(...)` loops.
- Sequencing: a message must be enqueued, then a doorbell raised, then an interrupt asserted; FIFO-full applies backpressure; masked channels suppress interrupts.
- Timing: every delay comes from the template's `timing_model`.
- Observability: IP-tagged logs (`[mailbox_ip] ...`) and a metrics dict the unit tests assert on.

7. How to add a new IP

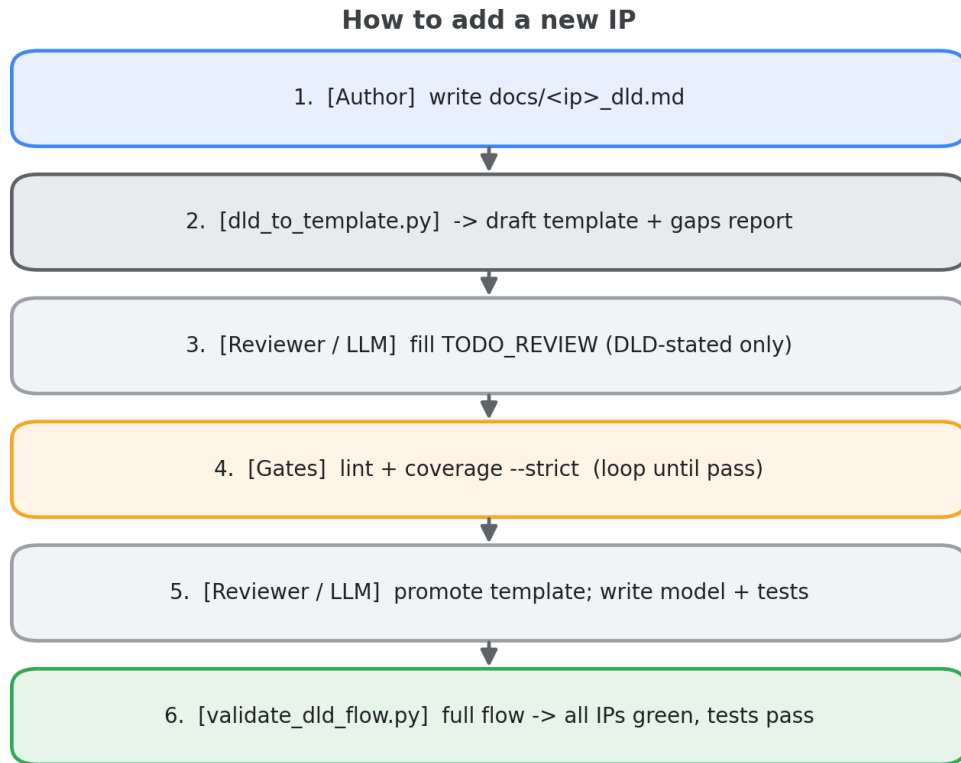


Figure 8. The add-an-IP loop, from DLD through review to a validated model.

- 1 Write docs/_dld.md (Purpose, Interfaces, FSMs+States, timing table, Open Items).
- 2 Run: `python tools\dld_to_template.py docs\_dld.md -> draft + gaps.`
- 3 Fill every TODO_REVIEW; resolve gaps using only DLD-stated behavior.
- 4 Check coverage --strict, then promote the draft to templates\template.yaml.
- 5 Implement src/ip_model_automation/.py and tests/test_.py (register the model in common.py + ip.py).
- 6 Run: `python tools\validate_dld_flow.py -> green.`

This is exactly the path used to add mailbox_ip and spi_master_ip: two brand-new DLDs went through extraction, review, modeling, and testing, and the whole 8-IP suite validated with 43 passing tests.

8. Summary

- We convert inconsistent, incomplete DLDs into validated SimPy models.
- A normalized template sits in the middle as the golden reference.

- Deterministic tools handle extraction, linting, coverage, and validation; a human/LLM fills the judgment gaps under a strict contract.
- Missing details are surfaced in a gaps report, never invented.
- Every stage has an automated gate, and one command (`validate_dld_flow.py`) proves the whole repo is consistent.